

Flux

Plateforme de réservation hôtelière
& de mise en relation locative



Rapport de conception

Architecture, modèle de données et workflows métier

Framework

Laravel 13.8

Version du document

1.0

Date

9 août 2026

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectifs du document	2
1.3	Périmètre	2
2	Analyse des besoins	3
2.1	Acteurs	3
2.2	Cas d'usage principaux	3
2.2.1	Module hôtellerie	3
2.2.2	Module logements	4
3	Architecture technique	5
3.1	Stack technologique	5
3.2	Organisation applicative	5
3.3	Services applicatifs	6
4	Modèle de données	7
4.1	Vue d'ensemble	7
4.2	Tables principales	7
4.3	Décisions de conception notables	8
5	Workflows métier	9
5.1	Inscription en deux étapes	9
5.2	Validation en cascade	9
5.3	Réservation hôtelière et paiement	9
5.4	Location de logement	10
6	Intégration du paiement AangaraaPay	11
6.1	Paiement direct	11
6.2	Effets métier à la confirmation	11
7	Interface & expérience utilisateur	12
7.1	Palette de couleurs	12
7.2	Typographie	12
7.3	Composants transverses	12

8	Sécurité	13
8.1	Contrôle d'accès	13
8.2	Validation en profondeur	13
8.3	Accès admin en lecture seule	13
9	Limites connues et perspectives	14
10	Conclusion	15

Chapitre 1

Introduction

1.1 Contexte

Flux est une plateforme web réunissant deux métiers distincts sous une même identité :

- la **réservation hôtelière** : recherche, consultation et réservation de chambres d’hôtel, avec paiement mobile ;
- la **mise en relation locative** : publication de logements (chambres, studios, appartements, villas, mini-cités) par des bailleurs et location par des clients, avec suivi de bail.

Quatre profils d’utilisateurs cohabitent sur la plateforme : **administrateur**, **hôtelier**, **bailleur** et **client**. Chacun dispose d’un espace dédié avec ses propres fonctionnalités, mais tous partagent le même socle applicatif.

1.2 Objectifs du document

Ce rapport décrit les choix de conception retenus pour construire Flux : architecture technique, modèle de données, workflows métier (inscription, validation, réservation, location, paiement) et principes d’interface. Il sert de référence pour la suite du développement et pour toute personne reprenant le projet.

1.3 Périmètre

Le document couvre la conception applicative telle qu’implémentée dans le code livré (migrations, modèles, contrôleurs, vues). Il ne couvre pas le déploiement infrastructure (serveurs, CI/CD), qui relève d’un document distinct.

Chapitre 2

Analyse des besoins

2.1 Acteurs

Acteur	Rôle sur la plateforme
Administrateur	Valide les comptes hôtelier/bailleur, les hôtels et les logements ; modère les avis ; consulte (sans modifier) l'ensemble des hôtels, logements, baux et réservations ; publie les actualités ; consulte les rapports financiers.
Hôtelier	Crée et gère ses hôtels (une fois validés par l'admin), leurs catégories de chambres, leur galerie photo, leurs réseaux sociaux et contacts de paiement ; traite les réservations reçues.
Bailleur	Crée et gère ses logements et mini-cités (une fois validés par l'admin) ; traite les demandes de location ; suit les baux en cours et leurs paiements ; valide les demandes de prolongation.
Client	Recherche et réserve des hôtels ou des logements ; paie en ligne (Mobile Money) ; suit ses réservations, ses locations et ses paiements ; laisse des avis.

2.2 Cas d'usage principaux

2.2.1 Module hôtellerie

1. Recherche d'un hôtel par destination, période, nombre de personnes.
2. Filtrage par nombre d'étoiles et note.
3. Consultation de la fiche hôtel (galerie, localisation, catégories de chambres, avis).
4. Réservation d'une chambre avec calcul du coût total en direct.
5. Paiement Mobile Money et confirmation de la réservation.
6. Suivi du séjour, puis réception d'un pro-forma à la clôture.

2.2.2 Module logements

1. Recherche d'un logement par type, catégorie, ville, prix.
2. Consultation de la fiche logement (galerie, équipements, localisation).
3. Prise de contact avec le bailleur en précisant une durée de bail souhaitée.
4. Validation de la demande par le bailleur et paiement initial (caution + durée minimum).
5. Suivi du bail et paiement des mensualités restantes au rythme du client.
6. Demande de prolongation (validée par le bailleur) ou fin de bail avec moratoire, puis remise en location automatique du logement.

Chapitre 3

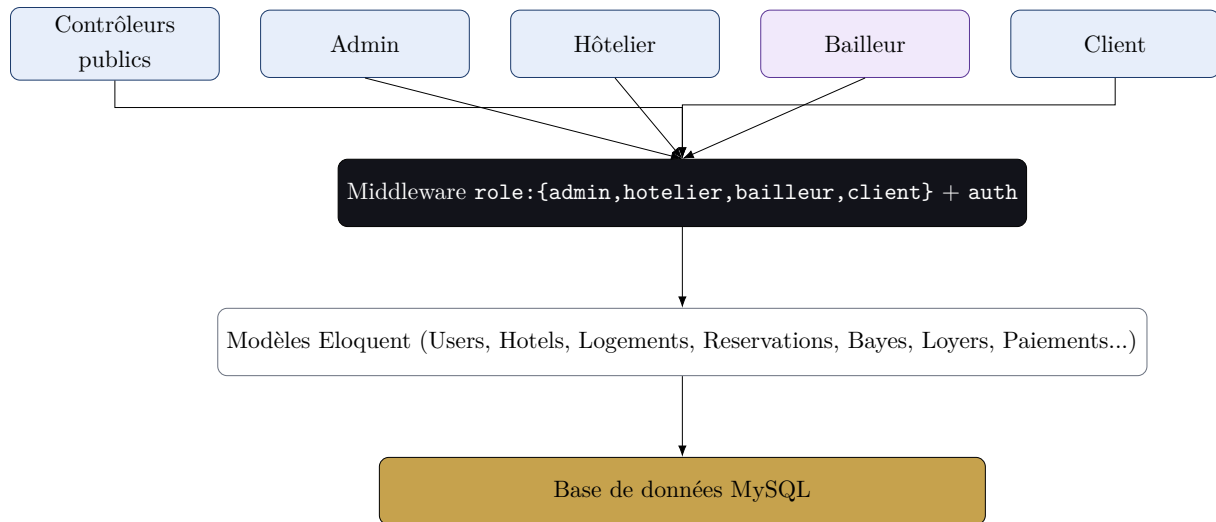
Architecture technique

3.1 Stack technologique

Composant	Choix technique
Framework backend	Laravel 13.8 (PHP 8.3+), architecture MVC classique
Base de données	MySQL
Frontend	Blade + Tailwind CSS v4 (configuration CSS-first), Alpine.js pour l'interactivité légère
Graphiques	Chart.js (courbes et camemberts dans les tableaux de bord)
Génération PDF	barryvdh/laravel-dompdf (pro-forma de réservation et de bail)
E-mail transactionnel	Composant Markdown Mail natif de Laravel
Paiement mobile	API AangaraaPay (MTN Mobile Money / Orange Money)
Tâches planifiées	Laravel Scheduler (<code>routes/console.php</code>)

3.2 Organisation applicative

L'application suit une architecture MVC standard, avec une segmentation nette des contrôleurs par espace :



Principe de séparation des rôles. Chaque groupe de routes est protégé par le middleware `role:xxx`, qui vérifie à la fois l'authentification et le rôle exact de l'utilisateur. Un hôtelier ne peut jamais accéder aux routes bailleur, et inversement.

3.3 Services applicatifs

Deux services encapsulent la logique technique transverse :

- `App\Services\AangaraaPayService` : abstraction de l'API de paiement (paiement direct, détection d'opérateur, vérification de statut) ;
- `App\Services\ProformaService` : génération des documents PDF pro-forma pour les réservations et les baux terminés.

Chapitre 4

Modèle de données

4.1 Vue d'ensemble

Le modèle de données s'articule autour de quatre blocs :

1. **Comptes** — `users` (admin, hôtelier, bailleur, client), avec statut de validation et vérification par code.
2. **Module hôtels** — hôtels, catégories de chambres, réservations, avis, favoris, actualités.
3. **Module logements** — logements, mini-cités, demandes de baye, baux, loyers.
4. **Transverse** — `photos` (galerie polymorphique) et `paiements` (polymorphique, relié à l'API AangaraaPay).

4.2 Tables principales

Table	Rôle
<code>users</code>	Compte unique pour les 4 rôles ; champs de vérification par code et de validation admin
<code>actualites</code>	Actualités affichées dans le carrousel de l'accueil (champ <code>ordre</code>)
<code>hotels</code>	Fiche hôtel, statut de validation admin
<code>categorie_chambres</code>	Catégories de chambres d'un hôtel (capacité, prix, équipements)
<code>reservations</code>	Réservation d'une chambre, statut, pro-forma PDF
<code>avis_hotels</code>	Avis client sur un hôtel, modéré par l'admin
<code>favoris</code>	Hôtels mis en favoris par un client
<code>minicites</code>	Regroupement de logements d'un même bailleur
<code>logements</code>	Chambre / studio / appartement / villa ; statut de disponibilité <i>et</i> de validation admin ; moratoire
<code>demandes_baye</code>	Demande initiale du client (« Contacter le bailleur »)
<code>bayes</code>	Contrat de location actif ou terminé, avec date de fin de moratoire

Table	Rôle
loyers	Échéances de paiement d'un bail (paiement initial ou mensualité libre)
prolongations	Historique des demandes de prolongation d'un bail
photos	Galerie polymorphique (hôtels, chambres, logements, mini-cités)
paiements	Transaction Mobile Money polymorphique (réservation ou loyer)

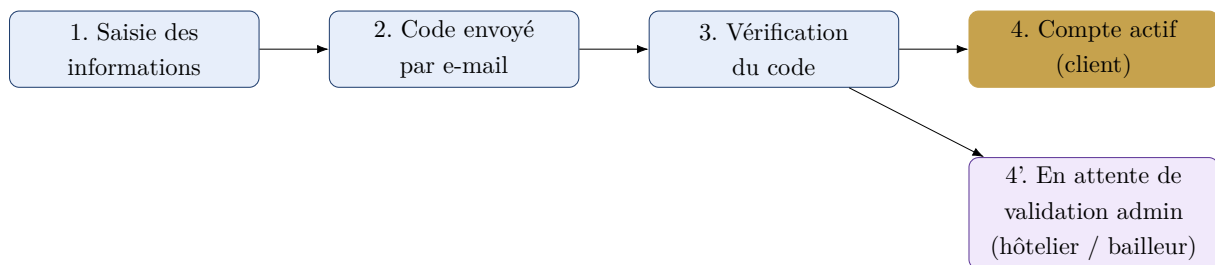
4.3 Décisions de conception notables

- **Polymorphisme** pour les galeries photo et les paiements, afin d'éviter la duplication de tables quasi identiques et de centraliser l'intégration avec l'API de paiement.
- **Double statut sur les logements** : `statut` (disponible / loué) est indépendant de `validation` (`en_attente` / `valide` / `rejete`) — un logement peut être disponible mais invisible tant qu'il n'est pas validé.
- **Dénormalisation contrôlée** : `hotels.note_moyenne` est recalculée à chaque approbation d'avis, pour permettre un tri rapide sans agrégation à la volée.
- **Mutateur Eloquent** sur `Logement` : le type « villa » force systématiquement la catégorie à « meublé », y compris si un autre champ est soumis — la règle est appliquée au niveau du modèle et non de la seule vue, ce qui la rend impossible à contourner.
- **Distinction paiement initial / mensualité libre** sur `loyers` (champ `paiement_initial`) pour représenter le plan de paiement en deux temps d'un bail.

Chapitre 5

Workflows métier

5.1 Inscription en deux étapes



Seuls les comptes **client** sont actifs immédiatement après vérification de l'e-mail. Les comptes **hôtelier** et **bailleur** déclenchent un e-mail à l'administrateur et restent bloqués jusqu'à validation ; la personne reçoit alors un e-mail de validation (accès débloqué) ou de rejet (avec motif, et possibilité de soumettre une nouvelle inscription).

5.2 Validation en cascade

Trois niveaux de validation admin coexistent et suivent le même schéma *demande* → *e-mail admin* → *décision* → *e-mail au demandeur* : le **compte** hôtelier/bailleur, l'**hôtel**, et le **logement**. Toute modification d'un hôtel ou d'un logement déjà validé le repasse en attente.

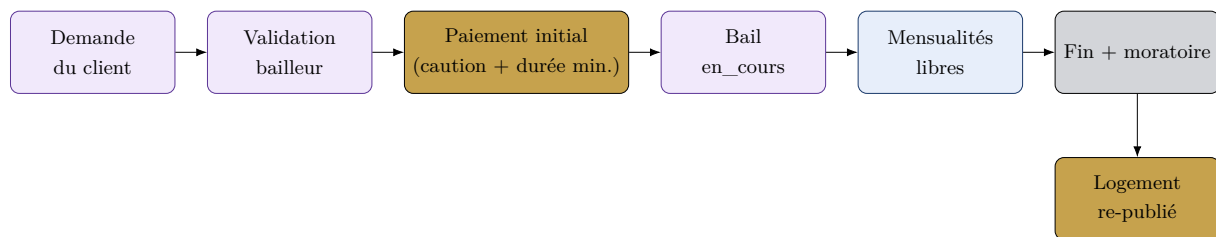
5.3 Réservation hôtelière et paiement

1. Le client choisit une catégorie de chambre et ses dates ; le coût total s'affiche en direct (calcul JavaScript côté formulaire).
2. À la validation, une réservation est créée au statut *en_attente* et le client est redirigé vers le paiement.
3. Le paiement direct (`AangaraaPayService::payerDirect`) envoie une notification Mobile

Money au téléphone du client ; l'écran d'attente interroge périodiquement le statut (*polling*) en parallèle du webhook serveur.

4. Une fois confirmé, la réservation passe au statut *confirmée*.
5. Une tâche planifiée quotidienne (`flux:terminer-sejours`) clôture les séjours dont la date de départ est passée : statut *terminée*, e-mail à l'hôtelier, pro-forma PDF généré et envoyé au client.

5.4 Location de logement



1. Le client indique une **durée souhaitée** (supérieure ou égale à la durée minimum du logement) en contactant le bailleur.
2. Le bailleur valide la demande : un bail est créé (statut *nouveau*), le logement passe en *loué*, et un **paiement initial** est généré, regroupant la caution et les mois de la durée minimum.
3. Une fois ce paiement confirmé, le bail passe en *en_cours*. Les mois suivants sont des mensualités indépendantes, payables par le client à son rythme.
4. Le client peut demander une **prolongation**, qui doit être validée par le bailleur ; la date de fin et le moratoire sont recalculés en conséquence.
5. À l'échéance (moratoire compris), une tâche planifiée quotidienne (`flux:traiter-baux`) clôture le bail : statut *termine*, logement remis en *disponible* (donc de nouveau visible sur le site), e-mail au bailleur, pro-forma PDF envoyé au client.

Rôle du moratoire. Paramétrable par logement (7 jours par défaut), il laisse un délai après la fin théorique du bail pour que le bailleur récupère le logement ou que le client obtienne une prolongation, avant que l'annonce ne redevienne publique.

Chapitre 6

Intégration du paiement AangaraaPay

6.1 Paiement direct

Flux utilise le flux de paiement « direct » de l'API AangaraaPay (documentation officielle : <https://aangaraa-pay.com/integrate-aangaraa-pay>), adapté aux cas où le numéro du client est déjà connu : aucune redirection vers une page tierce, le client reçoit directement une notification sur son téléphone pour approuver la transaction.

1. Détection automatique de l'opérateur (MTN ou Orange) à partir des préfixes du numéro camerounais.
2. Création d'un enregistrement `Paielement` (polymorphique) au statut `en_attente`.
3. Appel à l'API avec un identifiant de transaction unique et une `notify_url` pointant vers le webhook Flux.
4. Double confirmation : interrogation périodique du statut côté client (JavaScript) *et* réception du webhook côté serveur — le premier des deux qui aboutit déclenche la confirmation.

6.2 Effets métier à la confirmation

Le contrôleur central (`PaielementController::confirmerPaielement`) applique les effets selon le type d'objet payé :

- `Reservation` → statut *confirmée*;
- `Loyer` (paiement initial) → statut *payé*, activation du bail (*nouveau* → *en_cours*);
- `Loyer` (mensualité) → statut *payé*, recalcul de l'état de paiement global du bail.

Chapitre 7

Interface & expérience utilisateur

7.1 Palette de couleurs

La charte graphique distingue les deux univers de la plateforme tout en partageant une base commune : le **bleu** et l'**or** identifient le module hôtellerie, le **violet** et l'**or** le module logements.



7.2 Typographie

Les titres utilisent *Fraunces* (serif, registre « hôtellerie ») et le corps de texte *Inter* (sans-serif, lisibilité en interface), chargées via Google Fonts.

7.3 Composants transverses

- **Système d'icônes autonome** : composant Blade <x-icon> avec 25 tracés SVG dessinés à la main, sans dépendance à une librairie externe.
- **Responsive** : grilles adaptatives (1 colonne en mobile, jusqu'à 4 en desktop), navigation en menu hamburger, tableaux de bord à barre latérale rétractable.
- **Graphiques** : chaque tableau de bord (admin, hôtelier, bailleur) combine une courbe d'évolution sur 6 mois et un camembert de répartition, alimentés directement par les contrôleurs.
- **Filtres** : chaque liste de ressource dans les espaces connectés propose un filtre pertinent au contexte (statut, ville, type, note, période...).

Chapitre 8

Sécurité

8.1 Contrôle d'accès

Un middleware dédié (`EnsureRole`) vérifie à la fois l'authentification et l'appartenance au rôle attendu pour chaque groupe de routes. Les quatre espaces (`/admin`, `/hotelier`, `/bailleur`, `/mon-espace`) sont cloisonnés : aucune route n'est accessible à un rôle non habilité.

8.2 Validation en profondeur

Trois barrières successives protègent la visibilité publique du contenu généré par les professionnels :

1. vérification de l'e-mail (code à 6 chiffres, expiration 15 minutes) ;
2. validation du compte par l'administrateur (hôtelier / bailleur) ;
3. validation de chaque hôtel et de chaque logement par l'administrateur, y compris après modification.

8.3 Accès admin en lecture seule

L'administrateur dispose d'une vue complète sur les hôtels, logements, baux et réservations (module `Admin\SupervisionController`) sans disposer d'actions de modification sur ces ressources — la responsabilité de gestion reste entièrement du côté de l'hôtelier, du bailleur ou du client concerné.

Chapitre 9

Limites connues et perspectives

- Les tâches planifiées (clôture des séjours et des baux) nécessitent que `php artisan schedule:work` tourne en continu (ou une entrée `cron` en production) — sans cela, aucune clôture automatique n’a lieu.
- Le webhook de paiement doit être exposé publiquement pour être appelé par le serveur AangaraaPay (nécessite un tunnel type *ngrok* en développement local).
- Aucune notification SMS n’est envoyée à ce stade (uniquement des e-mails) ; une extension vers un fournisseur SMS local serait pertinente pour les utilisateurs sans accès régulier à leur boîte mail.
- Le bailleur ne reçoit pas encore de notification immédiate (au-delà de l’affichage dans son espace) lors d’une nouvelle demande de baye.

Chapitre 10

Conclusion

Flux couvre l'intégralité du parcours métier défini : inscription sécurisée en deux étapes, validation en cascade par l'administrateur, réservation hôtelière avec paiement Mobile Money direct, et location de logement avec plan de paiement flexible et gestion automatisée de la fin de bail via un système de moratoire. L'architecture retenue — séparation stricte des espaces par rôle, modèles polymorphiques pour les éléments transverses (galeries, paiements), et automatisation par tâches planifiées — doit permettre d'étendre la plateforme (nouveaux moyens de paiement, notifications SMS, statistiques avancées) sans remise en cause de la structure existante.